

UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería



Caso de Estudio 2

Diagnóstico del sistema de puntos MiMcDonald's Guatemala

Diego Linares

Andy Fuentes

Christian Echeverria

Diederich Solis

Machine Learning Engineering

Septiembre de 2026

Resumen de la situación

McDonald's Guatemala, operado por la franquicia maestra **McDonald's Mesoamérica** (126 restaurantes), lanzó el 27 de agosto de 2025 su programa de lealtad **MiMcDonald's**, que reemplazó a *Puntos McDelivery*. Quienes diseñaron el sistema ya no están en la empresa y no quedó documentación. Nos contrataron como consultores para diagnosticarlo y proponer mejoras en una semana.

El cliente no nos entregó datos, así que el reto fue doble: entender el sistema desde afuera y demostrar que lo entendimos. Lo trabajamos con **CRISP-DM**, fase por fase:

1. **Reconstruimos las reglas** del programa desde sus Términos y Condiciones oficiales: 22 reglas y 8 vacíos de documentación (McDonald's Guatemala, s.f.-a).
2. **Inferimos la arquitectura de datos actual** y la dibujamos.
3. Construimos una **réplica en miniatura** con arquitectura **medallion** (Bronze, Silver y Gold) en Databricks Free Edition, alimentada con **datos sintéticos** que respetan las reglas y traen 18 tipos de error inyectados a propósito.
4. Hicimos una **prueba funcional (POC)**: **39 de 39 pruebas OK**. El pipeline calcula los saldos de 5,150 clientes sin una sola diferencia contra una contabilidad independiente.
5. Integramos un **modelo de ML**: un recomendador de recompensas que acierta lo que el cliente canjea un 39 % más que recomendar lo más popular.

Hallazgos principales:

Hallazgo	Evidencia
El tope de 1,000 puntos diarios castiga a McDelivery, el canal que más gasta	Pierde el 39 % de sus puntos, contra un 17 % en caja
Breakage alto: muchos puntos vencen sin usarse	8.1 millones de puntos vencidos, el 44 % de lo acumulado por compras
La app no valida el mínimo de canje en McDelivery (Q50 / Q60)	33 canjes por debajo del mínimo
El catálogo no atiende a todos: desayuno y familias no tienen una recompensa accesible de su gusto	El café, la única recompensa barata de desayuno, salió del catálogo en junio
No hay verificación de identidad	150 cuentas duplicadas cobrando la bienvenida
No se publicó la tasa de conversión del programa anterior (1 pt/Q → 10 pts/Q)	Riesgo reputacional y contable

1. Cómo lo trabajamos

Fase CRISP-DM	Pregunta	En el repositorio
Entendimiento del negocio	¿Qué reglas tiene el programa?	docs/01
Entendimiento de los datos	¿Qué sistemas generan datos y con qué problemas?	docs/02, diagramas/
Preparación de datos	Réplica medallion	notebooks/00 a 03
Modelado	Recomendador de recompensas	notebooks/04
Evaluación	POC contra la hoja de respuestas	notebooks/05
Despliegue	Recomendaciones y este reporte	reporte/

Herramientas: Databricks Free Edition (Serverless, Unity Catalog, Volumes, tablas Delta y MLflow), PySpark, pandas, scikit-learn, Mermaid para los diagramas y GitHub conectado al workspace con Git folders.

2. Diagnóstico del sistema actual

Las reglas que reconstruimos

ID	Regla	Valor
R1–R2	Acumulación	10 puntos por cada Q1, redondeando hacia arriba (precios con IVA)
R4–R5	No acumulan	Donaciones y productos canjeados
R6–R7	Canales	Acumulan: mostrador, AutoMac, kioscos, McCafé, Centros de Postre y la app. No acumulan: apps de terceros, Call Center y WhatsApp
R8	Tope de acumulación	1,000 puntos por día
R9	Acreditación	Máximo 24 horas, lo que indica procesamiento por lotes nocturno
R10–R11	Vencimiento	365 días por lote, con aviso 30 días antes
R12–R13	Límites de canje	15,000 puntos por día y 3 ofertas por pedido
R14	Canje irreversible	Los puntos no se devuelven aunque se anule la orden
R15	Mínimo para canjear en McDelivery	Q50 en desayuno y Q60 en almuerzo y cena
R16	Catálogo	De 3,000 a 7,500 puntos; cambia sin previo aviso
R17	Bienvenida	Quesoburguesa al registrarse y 1,000 puntos con la primera compra
R20	Territorio	Solo Guatemala, aunque la plataforma se comparte con Honduras
R21	Migración	Los puntos anteriores “se convierten automáticamente”

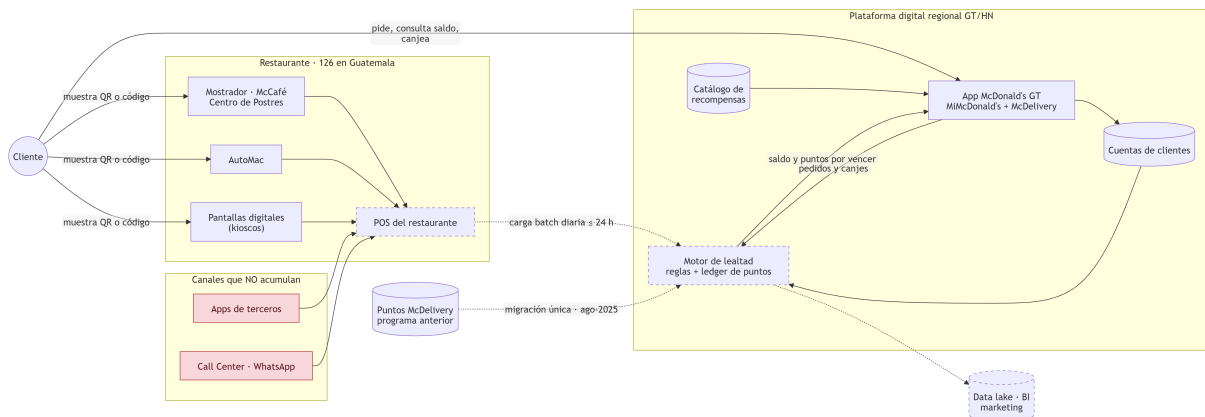
Lo que la documentación no dice

Un sistema que nadie documentó siempre tiene huecos. Los reportamos y, para poder avanzar, los resolvimos con supuestos explícitos que el cliente debe validar:

ID	Vacío	Supuesto en la réplica
H1	No se publica la tasa de conversión del programa anterior	×10 (conserva el valor en quetzales)
H2	No es explícito si el IVA suma puntos	Sí: se usa el total pagado
H3	No se dice qué pasa con los puntos de una orden anulada	No acumulan
H4	La acreditación en 24 h sugiere procesamiento batch	Pipeline diario
H5	El tope diario no define zona horaria ni orden de corte	Día de Guatemala, en orden cronológico
H6	“Dos correos = dos cuentas” y el teléfono es opcional	Se detectan cuentas de la misma persona
H7	La recompensa más barata exige Q300 y al menos 3 días de compra	Se analiza en Gold
H8	La nube del franquiciado no es pública; Google Cloud es del corporativo (McDonald’s Corporation, 2023)	Se reporta como supuesto

Arquitectura de datos actual (inferida)

Las compras en el restaurante pasan por el POS y llegan al motor de lealtad **por lotes**; las de la app entran directo. Un detalle importante: el POS solo conoce el código QR del cliente, no quién es. Borde punteado significa inferido; en rojo, los canales que no acumulan.



3. La réplica en miniatura

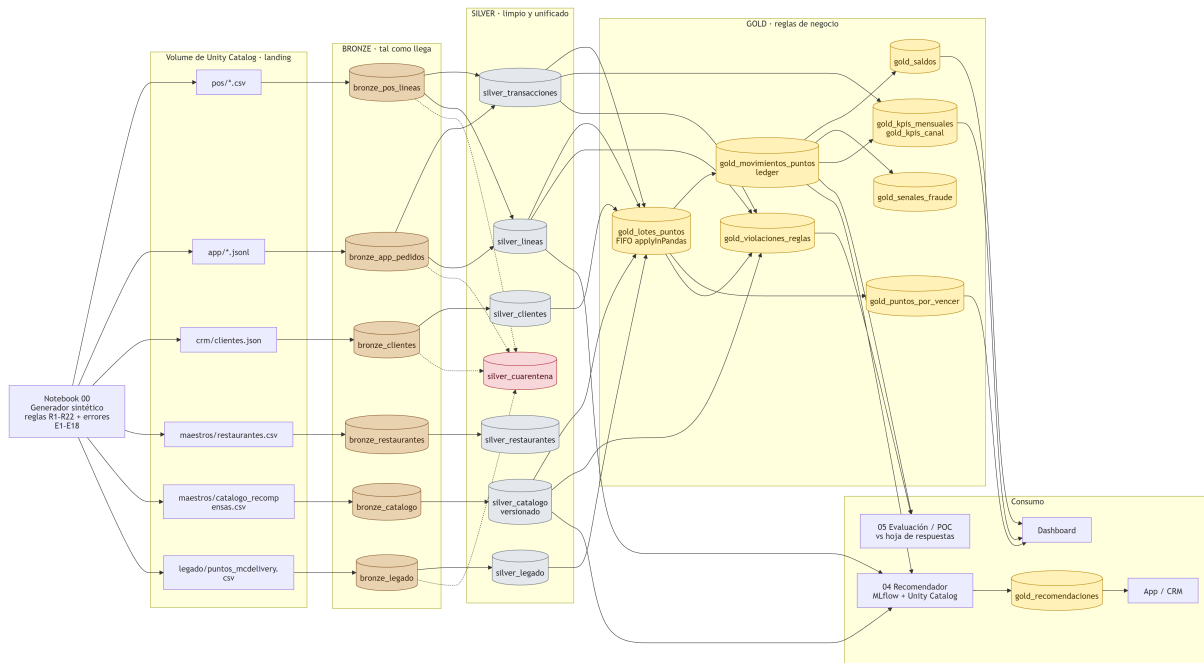
Datos sintéticos

Como no teníamos datos, el notebook `00_generador_datos.py` simula los **6 sistemas origen**, cada uno con su propio formato, igual que en la realidad: el POS manda CSV con hora local, la app manda JSON anidado con hora UTC y campos en inglés, y el CRM manda un solo arreglo JSON. Son 5,250 cuentas, 30 restaurantes y 13 meses de operación

(56,239 tickets), con **18 tipos de error** inyectados: archivos reenviados, canales mal escritos, precios con coma decimal, códigos QR inexistentes, eventos duplicados, cuentas duplicadas, saldos negativos, entre otros.

La decisión que más nos sirvió fue que el generador lleve **su propia contabilidad de puntos** en Python puro y la guarde como *hoja de respuestas* en un Volume aparte, que el pipeline nunca lee. Así, al final, pudimos comparar dos implementaciones independientes de las mismas reglas.

Arquitectura medallion



Capa	Qué hace	Tablas
Landing	Los archivos tal como los “envían” los sistemas	Volume
Bronze	Cada fuente como tabla, sin corregir nada (todo texto), más el archivo de origen y la fecha de ingesta	6 tablas
Silver	Limpia, tipa, deduplica y unifica POS y app en hora de Guatemala; lo irre recuperable va a cuarentena con su motivo	8 tablas
Gold	Aplica las reglas del programa : ledger de puntos, saldos, vencimientos, KPIs y violaciones	8 tablas

En nuestra opinión, lo más importante que aprendimos del medallion es la frontera entre capas: **Silver describe qué pasó y Gold decide qué significa**. Si mañana McDonald’s cambia la tasa de puntos, solo se toca Gold. Y como los puntos vencen por lote, **el saldo nunca se guarda: se calcula** sumando un libro de movimientos (*ledger*), igual que en contabilidad. Así cualquier saldo se puede auditar hasta la compra que lo originó.

4. Prueba funcional (POC) del sistema de puntos

Procedimiento

1. En Databricks: **Workspace** → **Create** → **Git folder** con la URL del repositorio.
2. Correr en orden, en Serverless: 00_generador_datos, 01_bronze, 02_silver, 03_gold, 04_recomendador y 05_evaluacion.
3. 05_evaluacion compara todo contra la hoja de respuestas y guarda el resultado en la tabla eval_resultados_poc.

Código clave

El tope diario sin ciclos. Si S es la suma acumulada de las compras del día, cada compra recibe $\min(1000, S) - \min(1000, S - \text{puntos})$, y eso cabe en una sola *window function*:

```
mismo_dia = (Window.partitionBy("customer_id", "fecha_gt")
               .orderBy("fecha_hora_gt", "transaccion_id")
               .rowsBetween(Window.unboundedPreceding,
                           Window.currentRow))

compras = (compras
           .withColumn("suma_dia", F.sum("puntos_calculados").over(mismo_dia))
           .withColumn("puntos_otorgados",
                       F.least(tope, F.col("suma_dia"))
                       - F.least(tope, F.col("suma_dia") - F.col("puntos_calculados"))))
```

Vencimiento por lotes (FIFO). Esto no cabe en SQL: lo que vence de un lote depende de lo que consumieron los canjes anteriores, y eso depende de lo que venció antes. Lo resolvimos cliente por cliente con `applyInPandas`, que Spark reparte en paralelo:

```
def procesar_lotes(pdf):
    lotes = []                                # [fecha, puntos_restantes]
    for fila in pdf.sort_values(["fecha_gt", "fecha_hora_gt"]).itertuples():
        vencer(fila.fecha_gt)                 # fecha del lote + 365 <= hoy
        if fila.puntos > 0:
            lotes.append([fila.fecha_gt, fila.puntos]) # abono: nace un lote
        else:
            costo = -fila.puntos               # canje: lo mas viejo
            for lote in lotes:
                usar = min(lote[1], costo)
                lote[1] -= usar; costo -= usar

lotes = movimientos.groupBy("customer_id").applyInPandas(procesar_lotes, ...)
```

Resultados: 39 de 39 pruebas OK

Parte	Qué se probó	OK
A. Reglas	Cada regla sobre todos sus casos reales, con el resultado esperado calculado aparte	12 / 12
B. Calidad de datos	Los registros detectados contra los inyectados, uno por uno	18 / 18
C. Saldos	Gold contra la hoja de respuestas, cliente por cliente	8 / 8
D. Recomendador	Recomendaciones contra el gusto real de cada cliente	1 / 1

Parte A, algunos ejemplos. $Q12.75 \times 10 = 127.5 \rightarrow 128$ puntos (R2). El cliente C002481 calculó 5,430 puntos en un día y recibió **1,000** (R8). De los canjes de Big Tasty, 158 costaron 7,000 puntos y 87 costaron 7,500, según la fecha del canje (R16). Hubo 1,378 tickets de Call Center, WhatsApp y apps de terceros con el cliente identificado, y ninguno acumuló (R7).

Parte B. No quisimos comparar solo conteos, porque dos errores distintos podrían compensarse. Comparamos los registros: para los 17 tipos de error, **0 faltantes y 0 falsas alarmas**. Es la misma lógica de *precision* y *recall* de un clasificador, pero aplicada a la limpieza de datos.

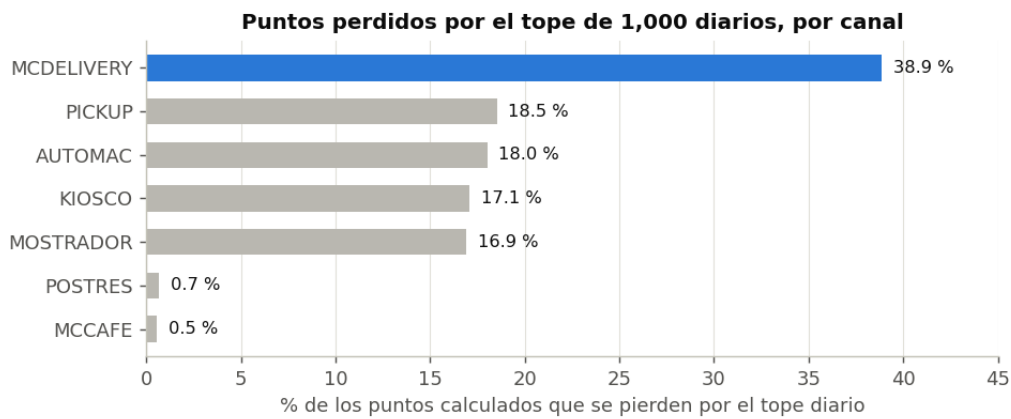
Parte C. Dos implementaciones escritas por separado (Python puro en el generador y Spark en Gold) llegan al mismo resultado:

Métrica	Total	Clientes con diferencia
Acumulados por compras	18,581,394	0 de 5,150
Bienvenida	4,336,000	0 de 5,150
Migrados del programa anterior	17,835,600	0 de 5,150
Canjeados	20,729,000	0 de 5,150
Vencidos	8,097,012	0 de 5,150
Perdidos por el tope diario	5,277,554	0 de 5,150
Saldo al 20 de septiembre de 2026	11,926,982	0 de 5,150

5. Hallazgos del diagnóstico

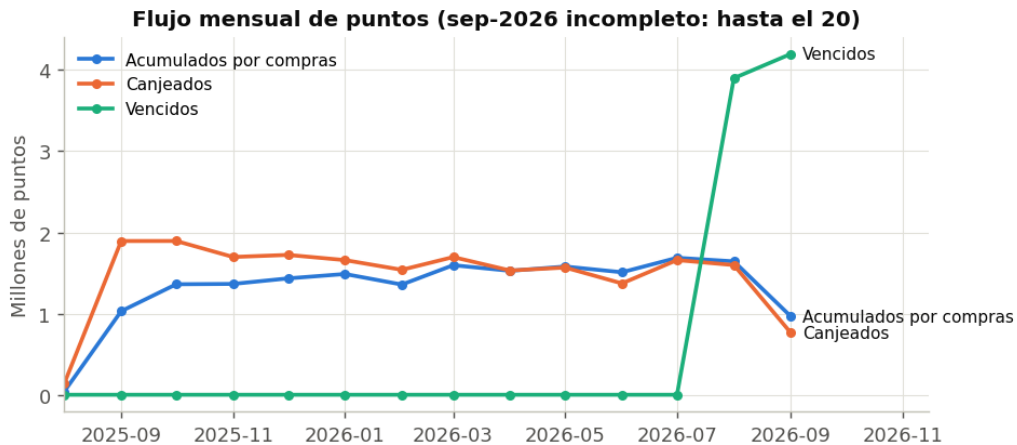
El tope diario castiga a los pedidos grandes

Con 10 puntos por quetzal y un tope de 1,000 puntos diarios, todo lo que se gaste arriba de Q100 en un día no genera puntos. En McDelivery el 67 % de los pedidos pasa de Q100, así que el canal de los clientes que más gastan es el peor tratado:



Los puntos migrados vencen en masa al año

Los puntos del programa anterior se abonaron en agosto y septiembre de 2025, y vencen todos juntos un año después. Para la empresa es un pasivo que desaparece; para los clientes más antiguos, una mala experiencia:



Otros hallazgos

Hallazgo	Dato	Riesgo
La app deja canjear en McDelivery sin llegar al mínimo (R15)	33 canjes	La regla no está implementada en la app
Terceros con pedidos grandes que el programa no ve	PedidosYa y Uber Eats: ticket de ~Q135	Se desconoce a los clientes de mayor gasto
Cuentas de la misma persona	150 cuentas	Abuso de la bienvenida
Saldo anteriores sin reclamar	252 correos	Pasivo sin fecha clara
Eventos corruptos en la app	55 sin cliente, restaurante o fecha	Compras que no acumulan y generan reclamos

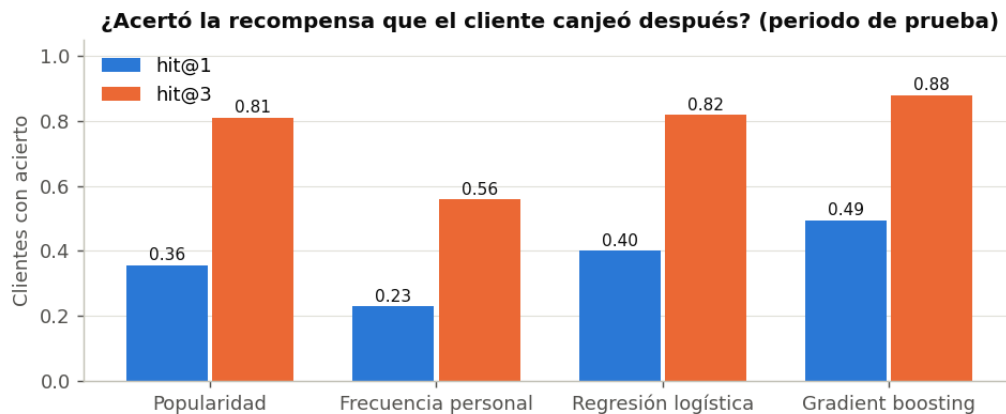
6. Propuesta: un modelo de ML dentro del pipeline

Recomendador de recompensas

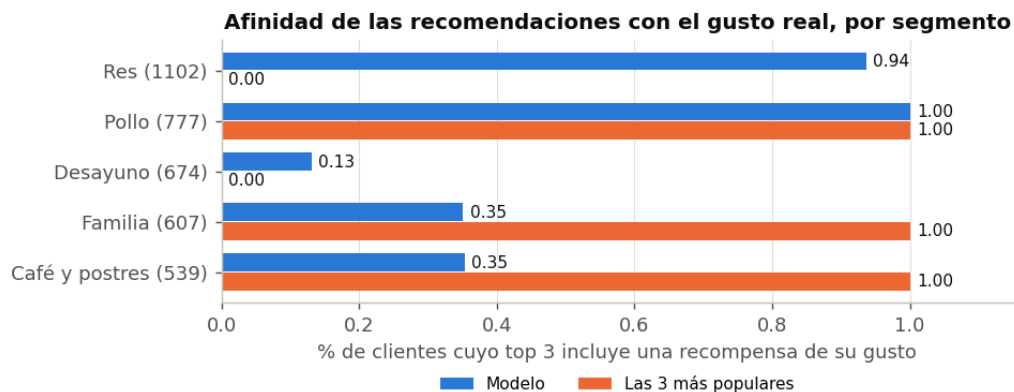
La pregunta es qué recompensa mostrarle a cada cliente en la app, que es lo que McDonald's hace a nivel corporativo con Dynamic Yield (Restaurant Technology News, 2026). El diseño es de **dos etapas**, como los recomendadores reales: primero se toman como candidatas las recompensas vigentes, y luego un modelo de *gradient boosting* las ordena combinando el gusto del cliente, el costo contra su saldo, la hora y la popularidad. El modelo queda registrado en MLflow y en Unity Catalog, y escribe `gold_recomendaciones` con un mensaje listo para la app, por ejemplo: “Te faltan 500 puntos para tu McFlurry”.

Tres decisiones de ingeniería que nos enseñaron algo:

- **Primero los baselines.** Con la primera versión de los datos, ningún modelo le ganaba a “recomendar lo más popular”. Faltaban gustos en los datos, así que generamos una segunda versión con perfiles de gusto por cliente.
- **Validación temporal.** Variables hasta el 20 de junio y etiqueta con los canjes del 20 de junio al 20 de septiembre: nada del futuro entra a las variables. Es la misma fuga de información que vimos en el Ejercicio 1 con `score` y `winner`.
- **El modelo evaluado es el que se despliega.** scikit-learn activa el *early stopping* solo cuando hay más de 10,000 filas, así que el modelo de producción se estaba entrenando distinto al evaluado. Lo fijamos explícitamente.

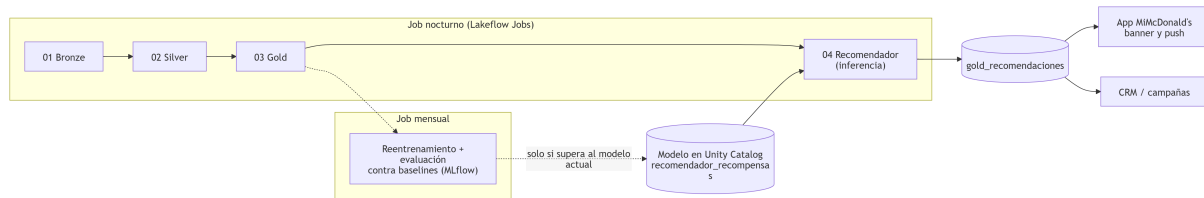


El modelo acierta la recompensa del banner principal (hit@1) un **39 % más** que recomendar lo más popular (0.495 contra 0.356). Además revisamos si aprendió los gustos *reales*, que nunca vio:



En total gana (0.62 contra 0.52), sobre todo en el segmento más grande (res). Pero lo que más nos llamó la atención es el segmento desayuno: **ni el modelo ni la lista popular lo atienden**. No es un problema del algoritmo sino del catálogo: las recompensas de desayuno cuestan 5,000 y 7,500 puntos, y la única barata, el café, salió en junio. También vimos que la variable que más pesa es el costo: el modelo predice bien lo que la gente canjea, que es lo barato. Si McDonald's quiere otro resultado, como más visitas o un ticket más alto, **el objetivo del modelo es una decisión de negocio**, no técnica.

Cómo se integra



Aspecto	Propuesta
Inferencia	Nocturna, después de Gold, con el saldo del día
Reentrenamiento	Mensual o cuando cambie el catálogo; el modelo nuevo solo reemplaza al actual si le gana a él y a los baselines
Monitoreo	hit@1 semanal con los canjes reales, clics del banner y diversidad de lo recomendado
Validación real	Prueba A/B: la mitad ve el catálogo actual y la otra mitad el recomendado, durante 30 días
Extensión con LLM	Un LLM (<code>ai_query()</code> en Databricks) redacta el mensaje, pero no decide qué recompensa ofrecer

7. Propuestas de optimización

Prioridad	Problema	Propuesta
Alta	El tope diario castiga a McDelivery	Subir el tope o excluir a McDelivery; como mínimo, avisarle al cliente en la app
Alta	La app no valida el mínimo de canje	Validar en la app y en el motor de lealtad, y auditar con la tabla de violaciones
Alta	Cuentas duplicadas cobran la bienvenida	Verificar el teléfono al registrarse (OTP)
Media	Breakage alto	Avisar 30 días antes de vencer (R11), con una recompensa recomendada
Media	Segmentos sin recompensa accesible	Una recompensa de desayuno de unos 3,000 puntos y una de Cajita Feliz
Media	Terceros invisibles para el programa	Integrar el código de lealtad en PedidosYa y Uber Eats
Media	Reglas sin documentar	Adoptar nuestra documentación como oficial y validar H1–H8 con el negocio
Técnica	Pipeline por lotes completos	Auto Loader para ingesta incremental, MERGE en Gold, movimientos de reverso para anulaciones tardías, <i>expectations</i> de Lakeflow y orquestación con Lakeflow Jobs

Conclusiones

1. **El sistema se puede documentar y auditar desde afuera.** Reconstruimos 22 reglas con fuentes públicas, encontramos 8 vacíos y los resolvimos con supuestos explícitos que el cliente debe validar.
2. **La arquitectura medallion es la adecuada para un programa de puntos.** La

acreditación en 24 horas ya indica procesamiento por lotes, y separar la evidencia (Bronze), la limpieza (Silver) y las reglas (Gold) permite auditar cualquier saldo y cambiar una regla sin tocar lo demás.

3. **El diseño del modelo de datos pesa más que la herramienta.** Como los puntos vencen por lote, el saldo no puede ser un número guardado: tiene que salir de un ledger.
4. **La réplica es confiable.** La POC pasó 39 de 39 pruebas: saldos idénticos en 5,150 clientes contra una implementación independiente y detección exacta de los 17 tipos de error.
5. **En nuestra opinión, lo más valioso del ML fue lo que reveló, más que lo que predijo.** El recomendador le gana al baseline, pero dejó ver que el problema del segmento desayuno está en el catálogo, y que qué optimizar es una decisión que le toca a McDonald's.
6. **Limitación:** los datos son sintéticos. La POC demuestra que el pipeline aplica bien las reglas; el siguiente paso es validar esas reglas con el cliente y correr el pipeline con sus datos reales.

Repositorio

<https://github.com/DiegoLinares11/MLOPS-Caso-de-estudio-2>

Contiene la documentación de cada fase ([docs/](#)), los diagramas en Mermaid ([diagramas/](#)), los seis notebooks de Databricks ([notebooks/](#)) y este reporte con sus datos e imágenes ([reporte/](#)).

Referencias

- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS.
- Databricks. (s.f.). *What is a medallion architecture?* <https://www.databricks.com/glossary/medallion-architecture>
- L'Express Franchise. (2026). *McDonald's llega a 126 restaurantes en Guatemala: quién es el dueño*. <https://lexpress-franchise.com/latam/ultimas-noticias/mcdonalds-llega-126-restaurantes-guatemala-quien-es-dueno/>
- McDonald's Corporation. (2023). *McDonald's announces new targets for development, loyalty membership, and cloud technology*. <https://corporate.mcdonalds.com/corpmcd/our-stories/article/mcd-announces-targets-development-loyalty-membership-cloud-tech.html>
- McDonald's Guatemala. (s.f.-a). *Términos y Condiciones App de McDonald's en Guatemala*. <https://mcdonaldsdigital.com/GML/public/app/gt/terminos-y-condiciones>
- McDonald's Guatemala. (s.f.-b). *MiMcDonald's Guatemala*. <https://www.mi.mcdonalds.com.gt/>

McDonald's Guatemala. (s.f.-c). *Puntos McDelivery*. <https://puntosmcdelivery.mcdonalds.com.gt/>

Periódico Digital Centroamericano y del Caribe. (2025). *McDonald's lanza "MiMcDonald's", su nuevo programa de lealtad para recompensar a sus invitados*. <https://nwsinamerica.com/pdcc/noticias/gerenciales/2025/mcdonalds-lanza-mimcdonalds-su-nuevo-programa-de-lealtad-para-recompensar-a-sus-invitados/>

Perspectiva. (2026). *McDonald's alcanza 126 restaurantes en Guatemala*. <https://perspectiva.gt/empresa/mcdonalds-alcanza-126-restaurantes-guatemala-2026/>

Restaurant Technology News. (2026). *McDonald's unifies data from nearly 220 million loyalty users as global AI strategy takes shape*. <https://restauranttechnologynews.com/2026/08/mcdonalds-unifies-data-from-nearly-220-million-loyalty-users-as-global-ai-strategy-takes-shape/>